The Journal of Open Source Software

Galv: Metadata Secretary for Battery Science

Brady Planden [1¶], **Matt Jaquiery** [2], **Martin Robinson** [2], **and David A. Howey** [1,3]

1 Department of Engineering Science, University of Oxford, Oxford, UK **2** Research Software Engineering Group, Doctoral Training Centre, University of Oxford, Oxford, UK **3** The Faraday Institution, Harwell Campus, Didcot, UK ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

The Galv project is a data and metadata management platform for battery science. Labs conducting battery research can use the platform to store and share (meta)data created from experimental electrochemical experiments. This data includes information about the electrochemical cell, the experimental, and the results. The platform provides a REST API for programmatic access to the data and metadata, in addition to a web interface for user interaction. The web interface acts provides additional functionality for importing and exporting data to widely supported data science languages, such as Python, Julia, and MatLab. This enables users to curate the experimental data within a single platform before completing advanced data analysis in their preferred language. The platform is designed to be extensible, allowing users to define their own metadata schemas and data types. This platform can be self-hosted or used as a cloud service enabling wider collaborations and sharing of experimental test data.

Statement of need

Galv is designed to address the need for a centralised data and metadata management platform for battery science. Battery research is a rapidly growing field, with many labs conducting experiments and generating data. This data is often stored in a variety of formats, making it difficult to share and compare results. By providing a centralised platform for storing and sharing data and metadata, Galv aims to make it easier for researchers to collaborate and build on each other's work. Futhermore, by defining a minimal structure for data, Galv can help researchers to ensure that their data is interoperable for meta-analysis and data science purposes.

Architecture

The Galv platform is composed of three key components, the `backend`, the `frontend`, and the `harvester`. The `backend` stores the corresponding (meta)data in parquet containers with a REST API that provides programmatic access for the frontend as well as third-party applications. The `frontend` is a web interface that allows users to interact, modify, and export the (meta)data stored in the platform. Finally, the `harvester` is a python package that can be used to automatically collect data and metadata from experiments and store it in the platform. The interaction of these three components is shown in Figure 1 below.

The Journal of Open Source Software

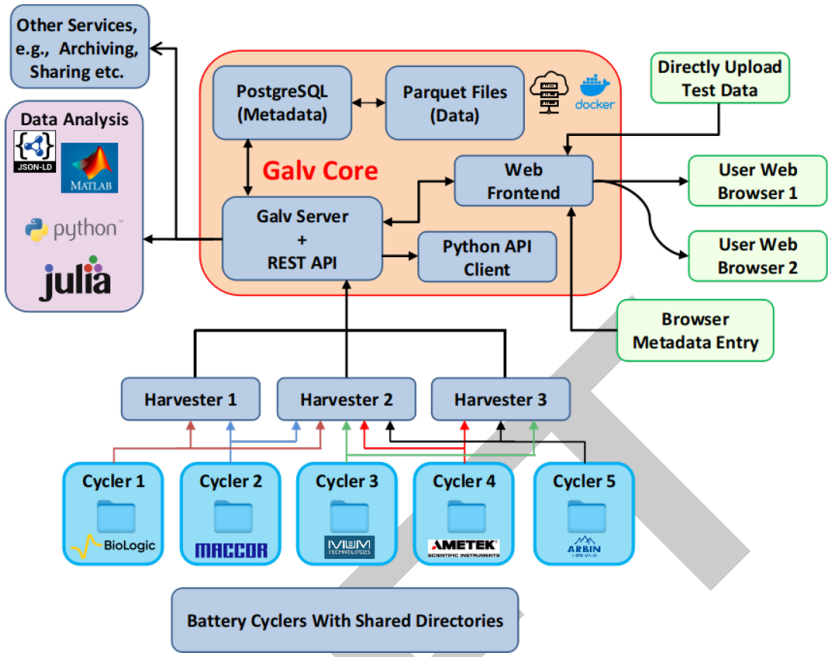

Figure 1: Galv's core architecture (backend) alongside data ingestion (harvesters), and the user web interface (frontend).

Galv is deployed as individual components with Docker images. These images can be built from the current state of development, or through images built during a Galv release and hosted on GitHub's container registry.

The backend server

The Galv backend is implemented with the Django framework (Foundation, 2024), with a corresponding REST API. This backend provide storage, authentication, and account management for the platform.

Items to discuss:
- Tweaks to the Django backend (superuser, example data)
- Parquet storage mechanism and performance over initial SQL database (i.e. splitting data and metadata for speed)
- Local storage and S3 Bucket

The `frontend` interface

The web-based frontend is implemented with the React framework and interfaces with the backend via the REST API. The frontend is the direct access for users' to the platform and is designed to provide the majority of the metadata curation. The frontend is structured to give end-users account management, split into individual user level, team-based, and laboratory. Each of these levels provides a structure tier for safe management of the underlying experimental data. The aim is to provide a structure that aligns with standard laboratory permissions and organisation.

Items to discuss:
- Metadata entry and organisation
- Time-series data rendering, application of CSV importing w/ header application (and storage for future events)
- Attachement of additional metadata in general object format to the underlying data (Image based datasheets, X-ray imaging, etc.)

The `Harvester` package

The harvester package is deployed through PyPI and provides the link between the experimental data acquisition within the laboratory and the Galv platform hosted either locally or cloud-based. Where possible, the harvesters parse the data files generated by the electrochemical cyclers and provide the backend a parquet converted container for storage. Currently, the supported proprietary data files include: Biologic, Maccor, and Ivium; however, data files stored in CSV format are parsed with the user providing mapping of the headers to the exportable object vectors in the frontend. Multiple harvesters can be setup on a single machine, with each monitoring a specified directory. Each of these harvesters is provided an API key on creation, which is generated from the specific backend instance and accessible from the frontend. As each harvester is connected to a specific instance of a Galv backend, multiple Galv instances can be used to monitor specific machines, which while not the main aim of the project, does showcase Galv's flexibility as a data acquisition platform.

Items to discuss:
- Harvester options
- The ability to regex the directory
- Galvani repository reference

Acknowledgements

We would like to acknowledge all contributors to `Galv` (link), and the funding bodies that supported this work: IntelLiGent, DigiBatt, Energy Superhub?, Digest.

References

Foundation, D. S. (2024). *Django: The web framework for perfectionists with deadlines* (Version 5.1.3). Django Software Foundation. https://github.com/django/django